Agentic AI for Automated RFP Response Generation

Title: SmartRFP: An Agentic AI System for Intelligent RFP Response Drafting using

SmartRFP – Product Requirement Document (PRD)

1. Product Overview

Product Name: SmartRFP – Agentic AI for Automated RFP Response Generation

Domain: Enterprise Pre-sales Pricing Aggregation

Audience: Sales staff, Sales Managers and External Project Stakeholders.

EY Scenario: Responding to a Large Enterprise Digital Transformation RFP. Assume a global client issues an RFP for a multi-year digital transformation program. The RFP may be extremely long, contain hundreds of requirements, and include technical, commercial, legal, and compliance sections. These often have very tight timelines for providing a response. In this scenario, the process can be sped up significantly via the incorporation of an agentic AI pipeline.

SmartRFP helps organisations automate the process of responding to RFPs (Request for Proposals). The system reads the RFP, retrieves relevant information from company documents, fetches current pricing data, and generates a proposal draft that can be reviewed and approved by a human.

General Problem Statement:

Organisations often receive detailed RFP (Request for Proposal) Documents from Clients and Vendors. Preparing a high-quality response is a time-consuming process because teams must:

- Understand the detailed requests
- Search past proposals for pricing and staff estimations
- Fetch the latest pricing
- Draft a structured response
- Manually get the response reviewed

The manual process causes:

- Slow turnaround time
- Inconsistent responses and outdated pricing
- Low reuse of existing enterprise knowledge

Our goal is to build an Agentic AI pipeline which automatically analyses an incoming RFP Document, searches the company information base for relevant knowledge points, and fetches accurate pricing from a static database.

Design Goal:

Build an Agentic AI-powered SmartRFP system that can automatically analyse incoming RFPs, retrieve relevant enterprise knowledge using RAG, fetch real-time pricing through APIs, generate a draft proposal response, and route it through a Human-in-the-Loop approval process before final submission.

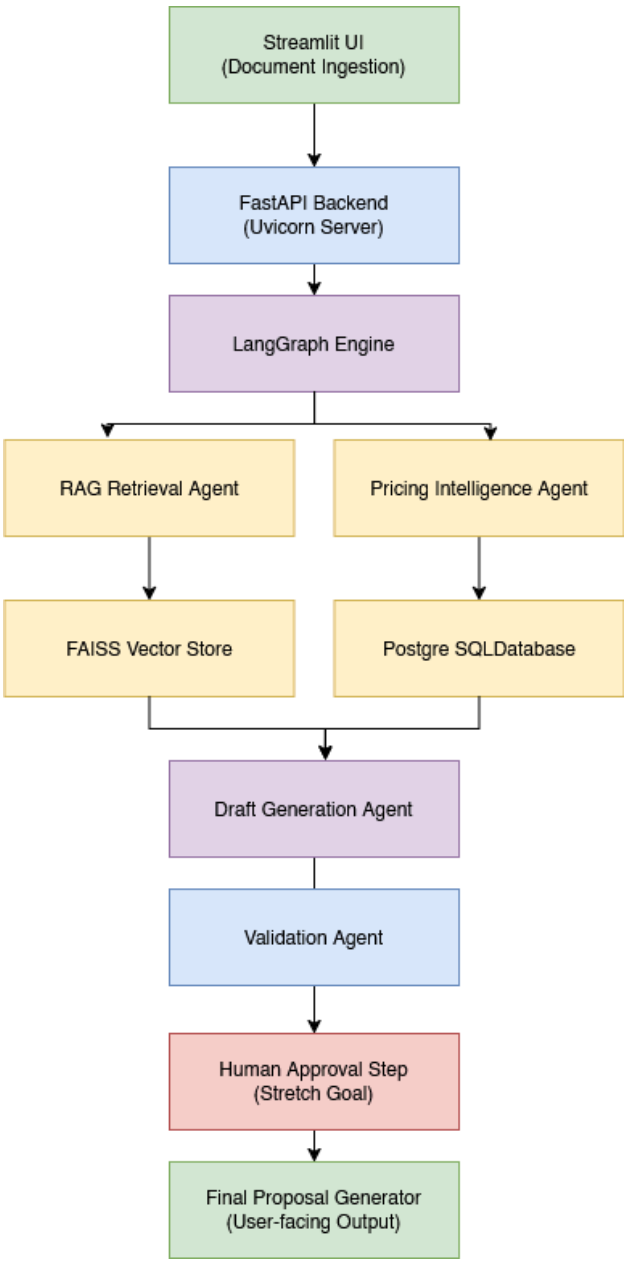
Functional Points to Cover:

- Read RFP automatically
- Understand requirements
- Search enterprise knowledge
- Fetch current pricing
- Draft proposal sections
- Ask humans only when necessary

Proposed Architecture Summary

After ingesting the input RFP document provided by the user, we enter a flow controlled by LangGraph, which calls two agents in parallel to fetch the core components of the developing response and the pricing for these components. After we have the necessary building blocks, we call the Document Synthesiser component, which returns a well-formatted response document to be sent to the customer. As a stretch goal, we will attempt to keep the option to have a human reviewer in this loop.

High-level Architecture Diagram



Detailed Architecture Design

Core Design

Agent 1: RAG Retrieval

Purpose

1. Ingesting enterprise documents into the RAG system
2. Converting documents into embeddings
3. Storing embeddings in a FAISS vector index
4. Retrieving relevant context for RFP questions

5. Providing knowledge-grounded responses to the Draft Generation Agent

Approach

We begin by ingesting the input document, splitting it into chunks and generating embeddings. Then, we use the embedding to either append to an existing FAISS index or create a new one. Then we use the RAG service to fetch relevant chunks from the existing company information base.

Design-Specific Choices

Parameter	Value	Purpose	Justification
CHUNK_SIZE	1000	Size of document chunks	To ensure relevant context is fetched in its entirety, as much as possible.
CHUNK_OVERLAP	150	Overlap between chunks	To ensure the overlap supplements any missing context in a single chunk.
TOP_K	5	Number of retrieved chunks	To ensure all relevant information is looked at.
SCORE_THRESHOLD	0.80	Minimum similarity score	To filter out irrelevant data, we only look at chunks at least 80% similar to the input

Agent 2: Pricing Intelligence

Purpose

This agent is responsible for fetching the latest commercial information from the enterprise database. The specific tasks achieved are:

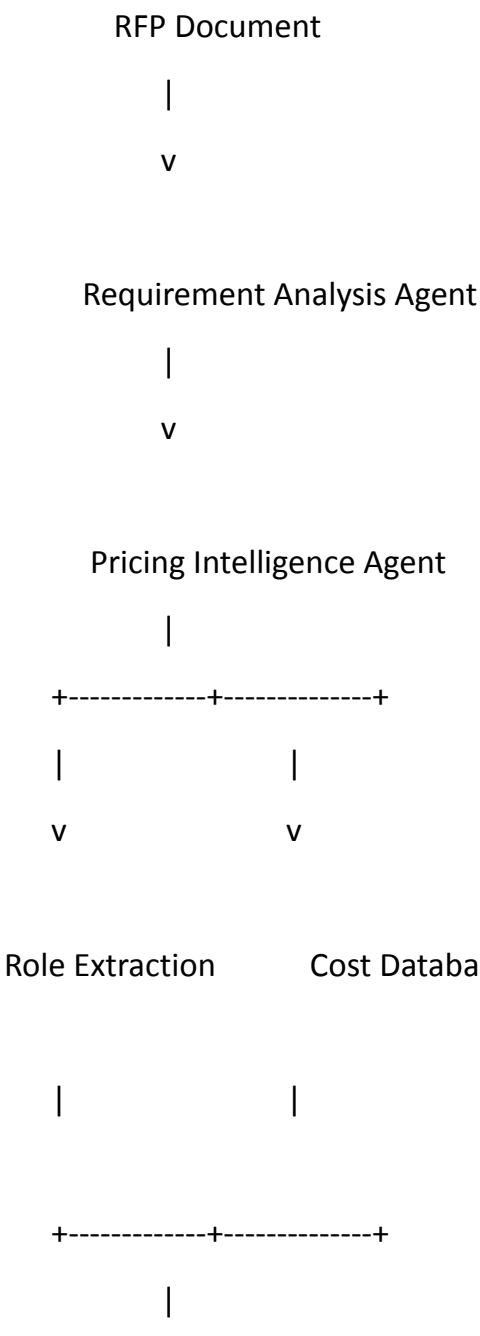
1. Extracting staffing requirements from the RFP
2. Mapping identified roles to enterprise resource catalogues
3. Retrieving current rates from PostgreSQL
4. Calculating daily, monthly, and annual costs
5. Producing structured commercial inputs for proposal generation

Approach

We extract the staffing requirements from the input documents, and specifically note the Roles that we need to find the pricing for. After we have the roles from the document, we fetch the costing details as accurately as possible by also extracting or imputing the number of people in these roles who are requested. Then we perform a lookup in our PostgreSQL database server and compute the totla cost, both at a header and a more granular level.

Design-specific Choices

Architecture Diagram



v

Cost Calculation Engine

|

v

Commercial Proposal Data

|

v

Draft Generation Agent

Draft Generator (LLM)

Purpose

The Draft Generation Agent is responsible for generating professional RFP response sections by combining:

1. RFP requirements
2. Enterprise knowledge retrieved through RAG
3. Commercial information from the Pricing Intelligence Agent
4. User instructions from Bid Managers or SMEs

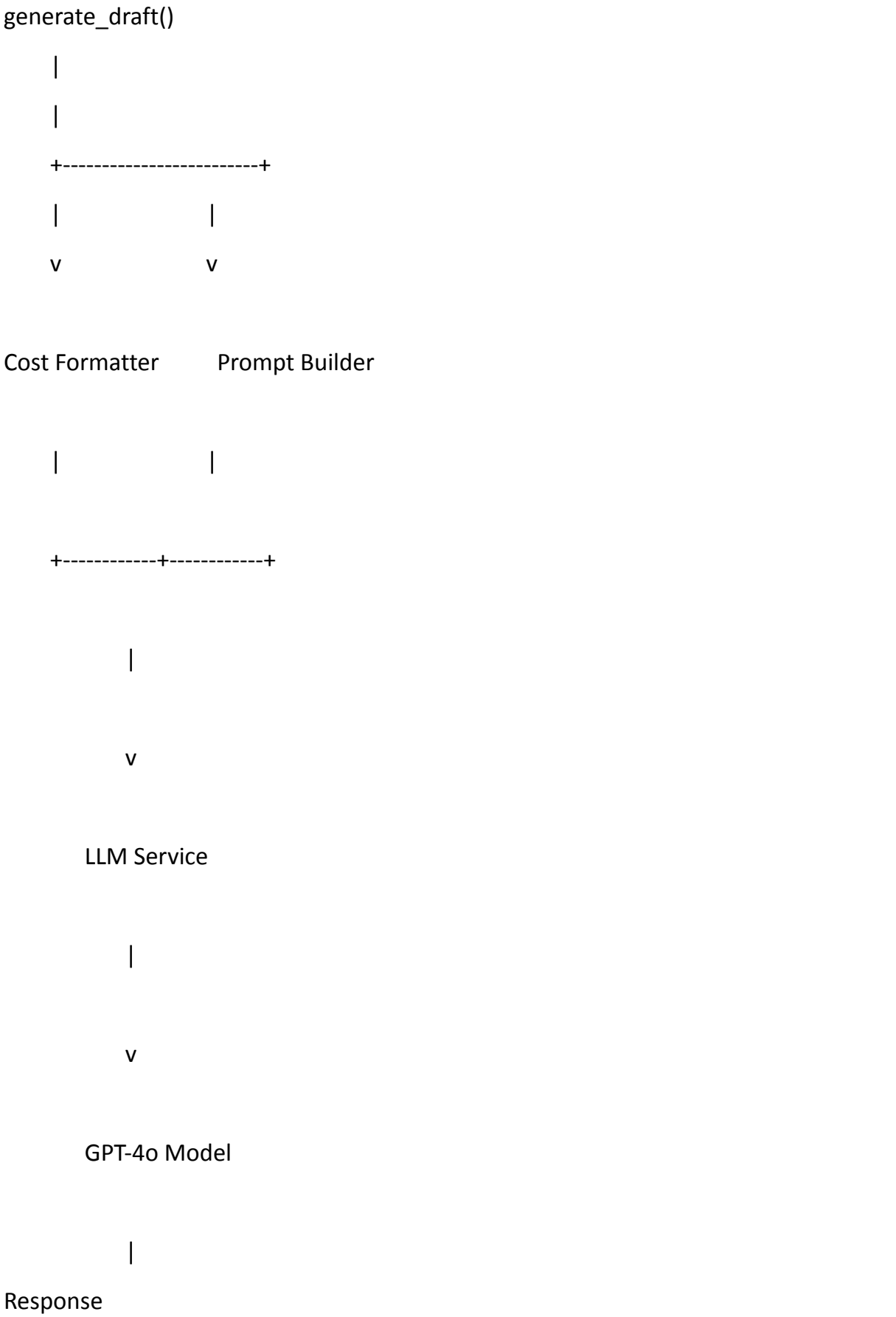
The agent acts as the final content synthesis layer before Human-in-the-Loop review.

Approach

In this step, we ingest the output from the previous two agents, as well as the original RFP input. Then we determine if the section to be generated is costing-related or not (to decide whether to show or hide the retrieved prices. Then, where costing is required, we convert the JSON into human-readable text. Finally, we assemble all the contextual information to build a single prompt for the final document generation. We also add generation constraints so that the model does not deviate from the task as hand or hallucinate. Then we initiate a call to the LLM (OpenAI GPT-4o) with the assembled prompt. We then pass this input to the validator engine.

Design-specific Choices

Architecture Diagram



User Interface

Purpose

Approach

Design-specific Choices

Validator Agent

Purpose

This is an important part of our complete workflow. Here, we ensure that the response generated by the model is accurate with respect to the prompt-with-context that was provided to it. We also check for compliance and legal violations at this stage, so that the output is clean and appropriate for the user.

Approach

We begin by implementing basic checks to ensure that the specific details, like the roles, per-role capacities, and costing as extracted by the two agents, are repeated accurately in the generated response.

We ensure that every step of the workflow is logged for better debugging in case of failure.

We also provide contingencies for the generator model being unavailable, and ensure that, in the cases where the model either does not return a response or returns a malformed output, we handle the issue gracefully, without throwing an unreadable error in the user's face.

We also measure hallucination via golden standard prompts and responses. We keep track of the latency for each prompt, alongside the length and complexity of the input provided.

We also capture the user's satisfaction with a feedback module, and we note an aggregation of that feedback as a qualitative metric.

Design-specific Choices

Architecture Diagram

Human-in-the-Loop (HITL)

Purpose

Approach

Design-specific Choices

Architecture Diagram

Evaluation Strategy

Unit Tests

TC-01: Role Extraction – Single Role Detection

Objective: Verify that the Pricing Agent correctly extracts a single role.

Input:
The project requires 1 Cloud Architect.

Expected Result:
Cloud Architect = 1

Module: Pricing Intelligence Agent

TC-02: Role Extraction – Multiple Roles Detection

Objective: Verify the extraction of multiple roles and counts.

Input:
1 Project Manager
2 Cloud Architects
4 Data Engineers

Expected Result:
Project Manager = 1
Cloud Architect = 2
Data Engineer = 4

Module: Pricing Intelligence Agent

TC-03: Role Normalisation

Objective: Verify aliases map to canonical roles.

Input:
Need 2 DBAs and 1 SRE.

Expected Result:
Database Administrator = 2
Site Reliability Engineer = 1

Module: Role Extraction Engine

TC-04: Cost Lookup – Exact Match

Objective: Verify exact role lookup from PostgreSQL.

Input:
Cloud Architect

Database Record:
Cloud Architect | $12,000/month

Expected Result:
Monthly Rate = $12,000

Module: Cost Lookup Engine

TC-05: Cost Lookup – Fallback Match

Objective: Verify fuzzy matching when an exact role does not exist.

Input:
Senior Cloud Architect

Database Record:
Cloud Architect

Expected Result:
System retrieves Cloud Architect pricing.

Module: Cost Lookup Engine

TC-06: Knowledge Base Ingestion

Objective: Verify uploaded documents are chunked and indexed.

Input:
5000-word proposal document

Expected Result:

- Document split into chunks
- Embeddings generated
- FAISS index updated successfully

Module: Knowledge Base Service

TC-07: Context Retrieval

Objective: Verify semantic retrieval returns relevant chunks.

Query:
Disaster recovery strategy

Expected Result:
Top-K relevant chunks returned from FAISS index.

Module: RAG Retrieval Agent

TC-08: Cost Formatting

Objective: Verify costing JSON converts to proposal-readable text.

Input:
Cloud Architect, Headcount = 2, Monthly Rate = $12,000

Expected Result:
Cloud Architect (x2): $12,000/month each

Module: Draft Generation Agent

Integration Tests

TC-09: Document Upload to Knowledge Base

Objective: Verify uploaded PDF is parsed and indexed.

Flow:
Upload PDF -> Parser -> Chunking -> Embedding -> FAISS

Expected Result:
Document becomes searchable in the knowledge base.

TC-10: RAG Retrieval Integration

Objective: Verify retrieval agent fetches enterprise knowledge.

Flow:
User Query -> Embedding -> FAISS Search -> Context Returned

Expected Result:
Relevant proposal sections and documents are returned.

TC-11: Pricing Agent Integration

Objective: Verify extracted roles are correctly mapped to database rates.

Flow:
RFP -> Role Extraction -> PostgreSQL Lookup -> Cost Calculation

Expected Result:
Accurate monthly, daily, and annual costs generated.

TC-12: Draft Generation Integration

Objective: Verify Draft Agent combines RFP content, retrieved knowledge, and pricing data.

Expected Result:
Generated response contains:

- RFP requirements
- Enterprise knowledge
- Pricing information
- Professional proposal language

TC-13: LangGraph Parallel Execution

Objective: Verify RAG Agent and Pricing Agent execute simultaneously.

Flow:
Requirement Analysis -> Parallel Execution (RAG + Pricing) -> Synthesis Node

Expected Result:
Both outputs are available before draft generation begins.

End-to-End Tests

TC-14: Complete Proposal Generation

Objective: Validate complete SmartRFP workflow.

Flow:
Upload RFP -> Requirement Analysis -> RAG Retrieval -> Pricing Agent -> Draft Generation -> Human Review

Expected Result:
Complete proposal draft generated successfully.

TC-15: Large Enterprise RFP Processing

Objective: Verify system handles large RFPs.

Input:
300-page enterprise RFP

Expected Result:

- No application crashes
- Successful processing
- Accurate retrieval results
- Draft generated within acceptable response time

TC-16: Missing Pricing Role

Objective: Verify fallback pricing mechanism.

Input:
Blockchain Governance Specialist

Expected Result:
Fallback estimated rate applied, and proposal generation continues successfully.

TC-17: Empty Knowledge Base

Objective: Verify behaviour when no documents are indexed.

Input:
Generate a proposal without uploaded enterprise documents.

Expected Result:
System displays knowledge base warning and continues generation using available RFP content.

TC-18: OpenAI Service Failure

Objective: Verify graceful handling of LLM failures.

Condition:
Invalid API key or OpenAI service unavailable.

Expected Result:

- User receives a meaningful error message
- Application remains operational
- Workflow does not crash

TC-19: Human-in-the-Loop Approval Workflow

Objective: Verify proposal approval process.

Flow:
Generated Draft -> Reviewer -> Approve

Expected Result:
Proposal status updated to Approved and available for final export.

TC-20: Human Rejection and Regeneration Workflow

Objective: Verify proposal regeneration after rejection.

Flow:
Generated Draft -> Reviewer Rejects -> Regenerate Draft

Expected Result:

- New draft generated
- Previous version retained in audit logs
- Status updated appropriately

Roadmap